NLP Assignment 2 Report

Muhammad Umer Siddiqui
ms08702@st.habib.pk
Dhanani School of Science and Engineering, Habib University
Karachi, Pakistan

Abdullah Shaikh
as09245@st.habib.pk
Dhanani School of Science and Engineering, Habib University
Karachi, Pakistan

Arqish Zaria
az09714@st.habib.pk
Dhanani School of Science and Engineering, Habib University
Karachi, Pakistan

Abstract

This report covers two sentiment classification tasks. For Question 2, we implemented a Naive Bayes classifier from scratch on a subset of the IMDB movie review dataset, reaching an F1-score of 0.816, and then improved it for the competition with negation handling, TF-IDF n-gram features, chi-square feature selection and Complement Naive Bayes, reaching a macro F1-score of 0.8861. For Question 3, we implemented a feedforward Artificial Neural Network from scratch with NumPy on a synthetic dataset of anonymized tokens.

Keywords

Natural Language Processing, Naive Bayes, Laplace Smoothing, TF-IDF, Chi-Square Feature Selection, Artificial Neural Network, Sentiment Classification, Backpropagation, One-Hot Encoding, ReLU, Sigmoid, Binary Cross Entropy

1 Question 2: Naive Bayes Classifier for Sentiment Analysis

1.1 Part 1: Naive Bayes from Scratch

1.1.1 Dataset Preparation. We used the IMDB Large Movie Review dataset. With a fixed random seed (42), we randomly selected 500 positive and 500 negative reviews from the training set, and 100 positive and 100 negative reviews from the test set. Each review was converted to lowercase, punctuation was removed, and English stop words (NLTK list) were filtered out. The cleaned text was then tokenized into words with NLTK's `word_tokenize`. The vocabulary was built from the unique words of the 1,000 training reviews only, and each review is represented by the frequency of each vocabulary word it contains.

1.1.2 Model. The classifier was implemented without any machine learning library. The prior of each class c is the fraction of training reviews in that class:

$$P(c) = \frac{N_c}{N} \tag{1}$$

The likelihood of each vocabulary word w given a class uses Laplace (add-one) smoothing, where $\text{count}(w, c)$ is the number of times w appears in training reviews of class c and $|V|$ is the vocabulary size:

$$P(w \mid c) = \frac{\text{count}(w, c) + 1}{\sum_{w' \in V} \text{count}(w', c) + |V|} \tag{2}$$

A test review d is assigned the class with the highest posterior probability, computed in log space. Words not in the vocabulary are skipped:

$$\hat{c} = \arg\max_c \left[\log P(c) + \sum_{w \in d} \text{count}(w, d) \cdot \log P(w \mid c) \right] \tag{3}$$

1.1.3 Results. The classifier was evaluated on the 200 test reviews. It achieved an accuracy of 82.0% and an **F1-score of 0.816** for the positive class (Table 1).

Table 1: Q2 Part 1: Evaluation Results on Test Set

Metric	Value
Accuracy	82.00%
Precision (Positive)	0.8333
Recall (Positive)	0.8000
F1 Score (Positive)	0.8163
Precision (Negative)	0.8077
Recall (Negative)	0.8400
F1 Score (Negative)	0.8235
Macro F1	0.8199

Table 2: Q2 Part 1: Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	80	20
Actual Negative	16	84

The confusion matrix (Table 2) shows that the classifier performs consistently across both classes. With a precision of 0.833 and recall of 0.800 for the positive class, the model is slightly more conservative in predicting positive reviews: it is usually right when it does, but misses a small fraction of actual positive reviews. The errors are well balanced (20 false negatives against 16 false positives), so the model did not develop a strong bias toward either class despite being trained on only 1,000 reviews.

1.1.4 Implementation Challenges.

- **Floating-Point Underflow:** Multiplying many small probabilities for long reviews causes numerical underflow, producing a probability of zero. We avoided this by working in log space with `math.log()`, summing the log-probabilities of tokens instead of multiplying raw probabilities.
- **Zero-Frequency Problem:** A word that never appears in one class during training would get a zero likelihood and cancel the entire review's score for that class. This was handled with Laplace smoothing (adding 1 to every word count).

1.1.5 Ideas for Improvement.

- **N-grams:** Bigrams or trigrams would let the model capture local context and modifiers, especially negations (distinguishing "good" from "not good").

- **Better Preprocessing:** Stemming or lemmatization would group variants of a word ("running", "runs", "ran" to "run"), shrinking the vocabulary and strengthening word statistics. Removing HTML tags such as `<br />` would also clean the reviews.
- **TF-IDF Weighting:** Replacing raw counts with TF-IDF would down-weight very common words and let the model focus on discriminative vocabulary.
- **More Training Data:** The model was trained on 1,000 of the 25,000 available training reviews. Using more data would improve vocabulary coverage and generalization.

1.2 Part 2: Naive Bayes Performance Competition

1.2.1 Approach. Our competition model follows the pipeline of Hamzah [1], who classified IMDB reviews by combining TF-IDF features, chi-square feature selection and Multinomial Naive Bayes, and showed that chi-square selection raised accuracy from 81.39% to 85.37%. We kept this TF-IDF and chi-square pipeline, used Complement Naive Bayes in place of Multinomial Naive Bayes, and did not use the AdaBoost step. Part 2 uses scikit-learn, while the Part 1 code is unchanged. Compared to the Part 1 baseline, we changed the following:

- **Training data:** all 25,000 training reviews (12,500 per class) instead of 1,000.
- **Text cleaning:** HTML tags are removed, text is lowercased and punctuation is removed. Stop words are no longer removed.
- **Negation handling:** after a negation word (e.g. "not", "no", "never" or a word ending in "n't"), every following word is prefixed with `NOT_` until the next punctuation mark, so "not good" produces the feature `NOT_good`.
- **Features:** TF-IDF over unigrams, bigrams and trigrams with sublinear term frequency, ignoring n-grams that appear in fewer than 3 reviews or in more than 95% of reviews. This gives 488,632 features.
- **Feature selection:** the top k features by chi-square score [1].
- **Classifier:** Complement Naive Bayes with smoothing parameter α.

1.2.2 Tuning. The number of selected features $k \in \{40000, 60000, 80000, 100000\}$ and the smoothing parameter $\alpha \in \{0.05, 0.1, 0.2, 0.3, 0.5\}$ were tuned with 5-fold cross-validation on the training set, using macro F1 as the score. The test set was not used for tuning. The best configuration was $k = 40000$ and $\alpha = 0.5$, with a cross-validation macro F1 of 0.8710.

1.2.3 Results. The final model was trained on all 25,000 training reviews and evaluated on all 25,000 test reviews (Tables 3 and 4). It reached a **macro F1-score of 0.8861**, compared to 0.8199 for the Part 1 baseline. The result can be reproduced by running `Q2/part2.py` with the `aclImdb` folder placed inside `Q2`.

Table 3: Q2 Part 2: Evaluation Results on Test Set

Metric	Value
Accuracy	88.61%
Precision (Positive)	0.8906
Recall (Positive)	0.8802
F1 Score (Positive)	0.8854
Precision (Negative)	0.8816
Recall (Negative)	0.8919
F1 Score (Negative)	0.8867
Macro Precision	0.8861
Macro Recall	0.8861
Macro F1	0.8861

Table 4: Q2 Part 2: Confusion Matrix

	Predicted Positive	Predicted Negative
Actual Positive	11003	1497
Actual Negative	1351	11149

Compared to Part 1, the errors stay balanced between the classes (1,497 false negatives against 1,351 false positives) while the error rate drops from 18.0% to 11.4%. In cross-validation, the smallest feature set ($k = 40000$) scored best for every value of α, and keeping more features generally lowered the score. This agrees with Hamzah [1], where a small chi-square selection (2,000 features) gave the best accuracy: many of the 488,632 n-grams add noise rather than sentiment information. A wider search toward smaller k and larger α, where the best values lay at the edge of our grid, may improve the result further.

2 Question 3: Artificial Neural Network for Sentiment Classification

2.1 Approach

2.1.1 Dataset Preparation. We loaded the synthetic dataset which contains 5000 training and 500 test samples. Each sample has a sentence made up of anonymized tokens like `word_1`, `word_2` and a binary label (0 or 1). We cleaned the text by converting to lowercase and removing punctuation, then tokenized each sentence by splitting on spaces. A vocabulary of 300 unique words was built from the training set only. Each sentence was then converted into a one-hot encoded vector of size 300.

2.1.2 Network Architecture. We implemented a simple feedforward ANN with one hidden layer from scratch using only NumPy:

- Input layer: 300 neurons (vocabulary size)
- Hidden layer: 128 neurons with ReLU activation
- Output layer: 1 neuron with Sigmoid activation

2.1.3 Forward Propagation. The forward pass computes:

$$Z_1 = X \cdot W_1 + b_1 \tag{4}$$
$$A_1 = \text{ReLU}(Z_1) \tag{5}$$
$$Z_2 = A_1 \cdot W_2 + b_2 \tag{6}$$
$$A_2 = \text{Sigmoid}(Z_2) \tag{7}$$

2.1.4 Loss Function. We used Binary Cross Entropy loss:

$$L = -\frac{1}{n} \sum [y \cdot \log(A_2) + (1 - y) \cdot \log(1 - A_2)] \tag{8}$$

2.1.5 Backpropagation. We computed gradients layer by layer using the chain rule.

Output Layer Gradients. Starting from the loss, we differentiate with respect to A_2:

$$\frac{\partial L}{\partial A_2} = -\frac{y}{A_2} + \frac{1 - y}{1 - A_2} \tag{9}$$

The derivative of Sigmoid is:

$$\frac{\partial A_2}{\partial Z_2} = A_2(1 - A_2) \tag{10}$$

Applying the chain rule:

$$\delta Z_2 = \frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} = \left(-\frac{y}{A_2} + \frac{1 - y}{1 - A_2}\right) \cdot A_2(1 - A_2) = A_2 - y \tag{11}$$

Since $Z_2 = A_1 \cdot W_2 + b_2$:

$$\delta W_2 = \frac{1}{n} A_1^T \cdot \delta Z_2 \tag{12}$$
$$\delta b_2 = \frac{1}{n} \sum \delta Z_2 \tag{13}$$

Hidden Layer Gradients. Propagating the error back through W_2:

$$\delta A_1 = \delta Z_2 \cdot W_2^T \tag{14}$$

The derivative of ReLU is:

$$\text{ReLU}'(Z_1) = \begin{cases} 1 & \text{if } Z_1 > 0 \\ 0 & \text{if } Z_1 \leq 0 \end{cases} \tag{15}$$

Applying the chain rule:

$$\delta Z_1 = \delta A_1 \times \text{ReLU}'(Z_1) \tag{16}$$

Since $Z_1 = X \cdot W_1 + b_1$:

$$\delta W_1 = \frac{1}{n} X^T \cdot \delta Z_1 \tag{17}$$
$$\delta b_1 = \frac{1}{n} \sum \delta Z_1 \tag{18}$$

Weight Updates. Weights were updated using gradient descent:

$$W = W - \alpha \cdot \delta W \tag{19}$$
$$b = b - \alpha \cdot \delta b \tag{20}$$

where $\alpha = 0.1$ is the learning rate.

2.2 Results

Table 5: Evaluation Results on Test Set

Metric	Value
Accuracy	52.60%
Precision (Positive)	0.5149
Recall (Positive)	0.9641
F1 Score (Positive)	0.6713
Precision (Negative)	0.7000
Recall (Negative)	0.0843
F1 Score (Negative)	0.1505
Macro F1	0.4109

Table 6: Confusion Matrix

	Predicted 0	Predicted 1
Actual 0	21	228
Actual 1	9	242

2.2.1 Confusion Matrix. The model predicts almost everything as class 1, as shown by the high recall for the positive class (0.9641) and near-zero recall for the negative class (0.0843). This imbalance is reflected in the large off-diagonal entry of 228 false positives against only 21 true negatives.

2.3 Challenges

The main challenge was the nature of the dataset. All 300 words of the vocabulary appear in both positive and negative sentences. This means there is no meaningful signal for the model to learn from, the words do not distinguish between the two classes. Secondaly, one-hot encoding produces sparse vectors where around 97% of values are zeros, and the model ends up predicting randomly. The loss stayed at 0.6931 throughout training.

2.4 Potential Improvements

- Using TF-IDF instead of one-hot encoding to give words different weights based on their discriminative power
- Adding more hidden layers to increase model capacity
- Using a real labeled sentiment dataset instead of synthetic anonymized tokens, where word choice actually correlates with the label.

2.5 Conclusion of Question 3

This part implemented a feedforward ANN from scratch using NumPy for binary sentiment classification. The network was trained correctly and the backpropagation derivations were verified, but the model could not learn because the synthetic dataset provides no discriminative signal: all vocabulary words appear with similar frequencies in both classes. The result is a model that collapses to predicting the majority class, achieving 52.60% accuracy with a loss fixed at 0.6931 throughout training.

3 Overall Conclusion

Both questions show that the representation of the text matters as much as the classifier. On real IMDB reviews, a from-scratch Naive Bayes classifier already reached an F1-score of 0.816, and better preprocessing, negation handling, TF-IDF n-gram features and chi-square feature selection raised the macro F1-score to 0.8861 while keeping a Naive Bayes model. On the synthetic dataset of Question 3, where every word appears in both classes, even a correctly implemented neural network could not learn, because the input features carried no information about the label.

References

[1] Muhamad Biki Hamzah. 2021. Classification of Movie Review Sentiment Analysis Using Chi-Square and Multinomial Naïve Bayes with Adaptive Boosting. *Journal of Advances in Information Systems and Technology* 3, 1 (April 2021), 67–74. https://journal.unnes.ac.id/sju/jaist/article/view/49098/19796